FinShare Backend — Pending Tasks for Abhilash

.NET 8 / ASP.NET Core / Dapper / SQL Server | April 2025

✅ What Is Already Done (Do NOT Redo)

Confirmed from the live API at https://fintrack.sysdatacenter.com — these are all working:

Area	Status
Auth (Phase 1)	✅ Register, Login, Verify Email, Resend Verification, Forgot Password, Reset Password
Password Policy	✅ PasswordValidator — strong password rules enforced on registration and password change
Email Service	✅ Graph API / SMTP — verification and reset emails working
Expenses Engine	✅ Full CRUD, multi-payer, attachments, account labels, personal expenses — 17 endpoints live
Groups	✅ Create, List, Get, Add Member, Remove Member — 5 endpoints live
Members	✅ Get, Update, Delete — 3 endpoints live
Settlement	✅ Suggest Payments, Get Summary, Mark Paid — 3 endpoints live
Subscription	✅ Plans, Current, Create, Update, Cancel — 5 endpoints live

📌 **Note:** AuthSecurity_v3.docx showed Email Verify, Forgot Password etc. as Missing — that was a spec document written BEFORE those features were built. All Phase 1 auth is confirmed live. Use FinShare_ForAbhilash_TrueStatus.docx for accurate current status.

🔐 PHASE 2 — Security Additions (DO THIS FIRST)

Reference document: FinShare_ForAbhilash_SecurityPhase2.docx

2A. Refresh Token System ⭐ IMMEDIATE NEXT TASK

Why this is needed:

- Right now when a user logs in, they get a JWT access token that expires after a short time (e.g. 15–60 minutes). When it expires, the user is logged OUT and must enter their email/password again. This is terrible user experience for a mobile app.
- A refresh token solves this. It is a long-lived token (7 days) stored in a secure httpOnly cookie (the browser/app cannot read it via JavaScript — only the server can). When the access token expires, the app silently calls /api/Auth/refresh, and the server issues a new access token WITHOUT asking the user to log in again.
- The refresh token is stored HASHED in the database (never plain text) — just like passwords. This means even if the database is breached, the tokens are useless.
- Token rotation: every time /refresh is called, the old refresh token is revoked and a brand new one is issued. This means a stolen token can only be used once before it becomes invalid.
- Without this feature, FinShare will frustrate users who get logged out constantly. This is the most impactful pending backend task.

SQL — New Table

🗄 NEW TABLE	`RefreshTokens`

– TokenId (uniqueidentifier PK, default NEWID())
– UserId (int FK → Members.MemberId)
– TokenHash nvarchar(500) — store SHA-256 hash of token, NEVER plain text
– ExpiresAt datetime2 — set to 7 days from issue (DATEADD(day, 7, GETUTCDATE()))
– IsRevoked bit default 0
– CreatedAt datetime2 default GETUTCDATE()
– RevokedAt datetime2 NULL — set when revoked
– ReplacedByToken nvarchar(500) NULL — hash of the new token that replaced this one (audit trail)
– INDEX: IX_RefreshTokens_UserId on (UserId) — for fast lookup of tokens by user

SQL — Stored Procedures

🗄 NEW SP	`sp_SaveRefreshToken`

– Params: @UserId int, @TokenHash nvarchar(500), @ExpiresAt datetime2
– Action: INSERT a new row into RefreshTokens
– Called when: user logs in, or after each token rotation (new token replaces old)

NEW SP	sp_GetRefreshToken

– Params: @TokenHash nvarchar(500)
– Action: SELECT * WHERE TokenHash = @TokenHash AND IsRevoked = 0 AND ExpiresAt > GETUTCDATE()
– Returns: the token row if valid, empty if revoked or expired
– Called when: /api/Auth/refresh endpoint is hit to validate the cookie

NEW SP	sp_RevokeRefreshToken

– Params: @TokenHash nvarchar(500), @ReplacedByToken nvarchar(500) NULL
– Action: UPDATE IsRevoked = 1, RevokedAt = GETUTCDATE(), ReplacedByToken = @ReplacedByToken
– Called when: token is rotated (revoke old), user logs out, or user revokes from another device

NEW SP	sp_CleanExpiredTokens

– Params: none
– Action: DELETE WHERE ExpiresAt < GETUTCDATE() OR IsRevoked = 1
– Called by: SQL Agent job — schedule to run nightly to keep the table small

C# — New Service

NEW FILE	Services/RefreshTokenService.cs

– GenerateRefreshToken() — creates 64 random bytes via RandomNumberGenerator, returns as base64 string
– HashToken(string token) — SHA-256 hash of the raw token — this is what gets stored in the DB
– CreateAndSaveAsync(int userId) — generates token, hashes it, calls sp_SaveRefreshToken, returns the RAW token (to set in cookie)
– ValidateAsync(string rawToken) — hashes the raw token, calls sp_GetRefreshToken, returns the UserId if valid or null if invalid/expired
– RevokeAsync(string rawToken, string newTokenHash = null) — hashes raw token, calls sp_RevokeRefreshToken

C# — Modify Auth Controller

MODIFY	Controllers/AuthController.cs

– POST /api/Auth/login — after issuing JWT access token, also call RefreshTokenService.CreateAndSaveAsync(userId). Set the raw token in an httpOnly cookie: Response.Cookies.Append("refreshToken", rawToken, new CookieOptions { HttpOnly = true, Secure = true, SameSite = SameSiteMode.Strict, Path = "/api/Auth/refresh", MaxAge = TimeSpan.FromDays(7) })
– POST /api/Auth/refresh — NEW ENDPOINT: (1) read Request.Cookies["refreshToken"] — return 401 if missing. (2) Call ValidateAsync() — return 401 if null. (3) Generate a new JWT access token for the returned UserId. (4) Call RevokeAsync(oldToken, newTokenHash) to revoke old. (5) Call CreateAndSaveAsync() for the new refresh token. (6) Set new cookie. (7) Return new access token in body.

– POST /api/Auth/revoke — NEW ENDPOINT: Requires [Authorize]. Read cookie, call RevokeAsync(), clear cookie with Response.Cookies.Delete("refreshToken").
– POST /api/Auth/logout — MODIFY EXISTING: also read the refresh token cookie, revoke it, and clear the cookie when logging out.

💡 **Tip:** The httpOnly cookie means JavaScript (and therefore XSS attacks) can NEVER read the refresh token. It is automatically sent by the browser on every request to /api/Auth/refresh. This is the most secure way to store it.

2B. WebAuthn / Biometric Login (LOW PRIORITY — do after 2A)

Why this is needed:

• WebAuthn lets users log in using Face ID, Touch ID, or a hardware security key instead of typing a password. This is especially valuable for a mobile expense app — users split bills frequently, so fast login matters.
• This is already in the security spec (SecurityPhase2.docx) but is NOT required for launch. It is low priority because all current users can log in normally. Implement only after refresh tokens are stable.
• Uses the Fido2NetLib NuGet package — a well-tested .NET library that handles all the cryptographic complexity. The 4-step flow (register-begin → register-complete → authenticate-begin → authenticate-complete) is standard WebAuthn.

- NuGet package to install: Fido2NetLib
- New table: WebAuthnCredentials (CredentialId, UserId, PublicKey, Counter, CreatedAt, DeviceName)
- New SPs: sp_SaveWebAuthnCredential, sp_GetWebAuthnCredentialByUserId, sp_DeleteWebAuthnCredential
- New endpoints: POST /api/Auth/webauthn/register-begin, /register-complete, /authenticate-begin, /authenticate-complete
- Full code for all SPs and the controller is in FinShare_ForAbhilash_SecurityPhase2.docx

👤 PHASE 3 — New User Features (After Phase 2A is done)

Reference document: FinShare_ForAbhilash_NewFeatures.docx

3A. Edit Profile & Profile Photo

Why this is needed:

- Currently there is no way for a user to change their display name, phone number, or profile photo after registering. This is a basic feature that every user expects — their name shows up on expense splits and settlement screens.
- Profile photos help members identify each other in group expense lists, especially in groups with many members who may not know each other's names.
- Photo upload must resize images to 200x200 pixels before saving. Without resizing, users could upload a 10MB photo and it would slow down every page that loads their avatar. SixLabors.ImageSharp does this in a single line of code.
- The GET endpoint is also used by the frontend to pre-fill the edit form with the user's current data.

SQL — Stored Procedures

🗄 NEW SP	`sp_GetUserProfile`

– SELECT MemberId, Name, Email, PhoneNumber, ProfilePhotoUrl, CreatedAt FROM Members WHERE MemberId = @UserId

– Called by: GET /api/User/profile to populate the edit profile form

🗄 NEW SP	`sp_UpdateUserProfile`

– Params: @UserId int, @Name nvarchar(100), @PhoneNumber nvarchar(20)

– UPDATE Members SET Name = @Name, PhoneNumber = @PhoneNumber WHERE MemberId = @UserId

– Return the updated row so the frontend can refresh without a second API call

– Do NOT allow email change here — email change requires re-verification (separate feature if needed later)

🗄 NEW SP	`sp_UpdateProfilePhoto`

– Params: @UserId int, @PhotoUrl nvarchar(500)

– UPDATE Members SET ProfilePhotoUrl = @PhotoUrl WHERE MemberId = @UserId

– Called after the file is saved to disk and the URL is known

C# — Controller

NEW FILE	Controllers/UserController.cs

– GET /api/User/profile — calls sp_GetUserProfile, returns UserProfileResponse. Requires [Authorize].
– PUT /api/User/profile — accepts UpdateProfileRequest (Name, PhoneNumber), calls sp_UpdateUserProfile. Requires [Authorize].
– POST /api/User/profile/photo — accepts IFormFile 'photo'. Validate it is an image (check ContentType). Use SixLabors.ImageSharp to resize to 200x200 JPEG. Save to wwwroot/uploads/profile/{userId}.jpg. Call sp_UpdateProfilePhoto with the URL. Requires [Authorize].

CONFIG	NuGet Package Manager

– Install: SixLabors.ImageSharp
– Usage: Image.Load(stream).Mutate(x => x.Resize(200, 200)).SaveAsJpeg(outputPath)

3B. Change Password

Why this is needed:

• Users need to be able to change their password from within the app — not just via the Forgot Password email flow. The forgot-password flow is for when they are LOCKED OUT. Change Password is for when they are LOGGED IN and want to update their password proactively.
• The current password must be verified before allowing the change. This prevents someone who briefly has access to an unlocked phone from changing the password and locking out the real owner.
• The new password must still pass through PasswordValidator (the same rules used at registration) to keep security consistent.
• BCrypt hashing: always hash the new password with a fresh salt before storing — never store plain text.

NEW SP	sp_ChangePassword

– Params: @UserId int, @NewPasswordHash nvarchar(200)
– IMPORTANT: Do NOT verify the current password in SQL. Do it in C# using BCrypt.Verify(currentPlain, storedHash) FIRST — only call this SP if verification passes.
– UPDATE Members SET PasswordHash = @NewPasswordHash WHERE MemberId = @UserId

MODIFY	Controllers/UserController.cs

– PUT /api/User/change-password — accepts ChangePasswordRequest { CurrentPassword, NewPassword, ConfirmNewPassword }
– Step 1: Fetch the user's current PasswordHash from DB (simple SELECT by UserId)
– Step 2: BCrypt.Verify(request.CurrentPassword, currentHash) — if false, return 400 'Current password is incorrect'
– Step 3: Run NewPassword through PasswordValidator — return 400 with errors if weak

– Step 4: BCrypt.HashPassword(request.NewPassword, workFactor: 12)
– Step 5: Call sp_ChangePassword with the new hash
– Requires [Authorize]

3C. Transaction History

Why this is needed:

• Currently a user can see expenses inside a specific group, but there is no single place to see ALL their financial activity across ALL groups. This is a major gap — users regularly ask 'how much have I spent this month?' or 'when did I pay that back?'
• The history combines three types of records: (1) Expenses the user paid for, (2) Settlements the user paid to someone, (3) Settlements someone paid to the user. This gives a complete financial picture in one scrollable list.
• The UNION ALL approach in a single stored procedure is the correct design — it is efficient, keeps the logic in one place, and allows filtering and sorting across all types at once.
• The summary endpoint (TotalSpent, TotalToReceive, TotalToPay) powers the 3-card dashboard on the frontend. Without this endpoint, the frontend cannot show those summary numbers.
• Pagination is essential — a user with months of expense history could have hundreds of transactions. OFFSET/FETCH keeps responses fast.

NEW SP	`sp_GetTransactionHistory`

– Params: @UserId int, @GroupId int NULL (optional filter), @Type nvarchar(20) NULL (optional: 'Expense'/'SettlementPaid'/'SettlementReceived'), @PageNumber int = 1, @PageSize int = 20
– Uses UNION ALL of three SELECT statements:
– (1) Expenses WHERE PayerMemberId = @UserId
– (2) Settlements WHERE PayerMemberId = @UserId AND IsPaid = 1
– (3) Settlements WHERE PayeeMemberId = @UserId AND IsPaid = 1
– Each row must return: TransactionId, Type, Amount, Description, GroupName, TransactionDate, OtherPartyName
– Wrap with outer SELECT that applies @GroupId and @Type filters, then ORDER BY TransactionDate DESC
– Use OFFSET (@PageNumber-1)*@PageSize ROWS FETCH NEXT @PageSize ROWS ONLY for pagination
– Also return TotalCount (for frontend to show 'Page 1 of 5')

NEW SP	`sp_GetTransactionSummary`

– Params: @UserId int
– Returns three values in one row:
– TotalSpent: SUM of all expenses where user is payer (all time or current month — decide with Manus)
– TotalToReceive: SUM of pending settlement amounts where user is the payee
– TotalToPay: SUM of pending settlement amounts where user is the payer
– This is a fast aggregation query — no pagination needed

NEW FILE	Controllers/TransactionController.cs

– GET /api/Transaction/history — query params: groupId?, type?, pageNumber (default 1), pageSize (default 20). Calls sp_GetTransactionHistory. Requires [Authorize].

– GET /api/Transaction/summary — no params. Calls sp_GetTransactionSummary. Returns TotalSpent, TotalToReceive, TotalToPay. Requires [Authorize].

3D. Notifications

Why this is needed:

• Currently when someone adds an expense to a group, the other members have no idea until they open the app and manually check. There is no notification, no badge, no alert. This breaks the core use case of a shared expense app.

• Notifications solve this: when an expense is added, everyone in the group gets a notification. When a settlement is marked as paid, the payee is notified. When someone joins a group, the new member is notified.

• The Notifications table stores these records. The frontend polls GET /api/Notification to check for new ones, shows a badge count (unread), and lets users mark them as read.

• The IsRead flag and TotalUnread count are critical — they power the notification bell badge on the frontend that shows '3' when there are unread items.

• Triggering notifications is done by calling sp_CreateNotification from within the existing controllers — it is just one extra DB call after the main action completes. There is no separate messaging service needed at this stage.

NEW TABLE	Notifications

– NotificationId int PK IDENTITY

– UserId int FK → Members.MemberId — the user who should receive this notification

– Type nvarchar(50) — 'ExpenseAdded', 'SettlementRequested', 'SettlementCompleted', 'MemberJoined'

– Title nvarchar(200) — short heading, e.g. 'New expense in Europe Trip'

– Message nvarchar(500) — detail, e.g. 'Abhilash added Dinner €45.00'

– IsRead bit default 0 — 0 = unread (shows in badge count), 1 = read

– RelatedEntityId int NULL — e.g. the ExpenseId or SettlementId so frontend can link directly to it

– CreatedAt datetime2 default GETUTCDATE()

– INDEX: IX_Notifications_UserId on (UserId) — fast lookup for all notifications of a user

NEW SP	sp_GetNotifications

– Params: @UserId int, @OnlyUnread bit = 0, @PageNumber int = 1, @PageSize int = 20

– SELECT notifications for user, ORDER BY CreatedAt DESC

– Also return TotalUnread (SELECT COUNT(*) WHERE UserId = @UserId AND IsRead = 0) as a second result set

NEW SP	sp_MarkNotificationRead

– Params: @NotificationId int, @UserId int
– UPDATE IsRead = 1 WHERE NotificationId = @NotificationId AND UserId = @UserId
– IMPORTANT: include the UserId check — otherwise User A could mark User B's notification as read

NEW SP	sp_MarkAllNotificationsRead

– Params: @UserId int
– UPDATE IsRead = 1 WHERE UserId = @UserId AND IsRead = 0
– Called when user taps 'Mark all as read'

NEW SP	sp_CreateNotification

– Params: @UserId int, @Type nvarchar(50), @Title nvarchar(200), @Message nvarchar(500), @RelatedEntityId int NULL
– INSERT into Notifications
– This SP is called from other controllers — not directly by the user

NEW FILE	Controllers/NotificationController.cs

– GET /api/Notification — returns paged list + TotalUnread count. Requires [Authorize].
– PUT /api/Notification/{id}/read — marks single notification as read. Requires [Authorize].
– PUT /api/Notification/mark-all-read — marks all as read. Requires [Authorize].

Where to Trigger Notifications — Add to Existing Controllers

After the main action succeeds, call sp_CreateNotification (via a helper method) for each affected user:

- ExpensesController → AddExpense: loop through all group members EXCEPT the payer, call sp_CreateNotification for each. Title: 'New expense in [GroupName]'. Message: '[PayerName] added [Description] — [Amount]'. Type: 'ExpenseAdded'. RelatedEntityId: new ExpenseId.
- SettlementController → MarkAsPaid: call sp_CreateNotification for the payee. Title: 'Payment received'. Message: '[PayerName] marked your settlement of [Amount] as paid'. Type: 'SettlementCompleted'. RelatedEntityId: SettlementId.
- GroupController → AddMember: call sp_CreateNotification for the new member. Title: 'You joined a group'. Message: 'You have been added to [GroupName]'. Type: 'MemberJoined'. RelatedEntityId: GroupId.

Tip: Create a small NotificationHelper.cs service with a single method SendAsync(int userId, string type, string title, string message, int? relatedId = null). This keeps each controller clean — just one line per trigger instead of duplicating the DB call code everywhere.

🏦 PHASE 4 — Bank Details / IBAN Storage (After Phase 3)

Reference document: FinShare_ForAbhilash_BankDetails.docx — full EncryptionService.cs code, all SPs, and controller methods

Why Bank Details Are Needed

Why this is needed:

- FinShare calculates who owes whom and creates settlement records. But right now, when User A needs to pay User B, there is no way for User A to find User B's bank account number inside the app. They have to ask for it over WhatsApp, which defeats the purpose of a digital expense app.
- The bank details feature lets each member save their IBAN once. When User A is marked as the payer in a settlement, they can tap a button inside the settlement screen to see User B's (the payee's) bank details and transfer directly.
- Security is critical here. IBANs must NEVER be stored as plain text in the database. If the database is ever breached, all bank account numbers would be exposed. AES-256 encryption solves this — the IBAN is encrypted before INSERT and decrypted only when needed.
- Access control is equally important. Only the PAYER of a specific settlement can see the PAYEE's IBAN for that settlement. Nobody else — not other group members, not even group admins — can retrieve someone else's bank details. This is enforced by sp_GetMemberBankDetailsForSettlement which checks the caller IS the payer before returning anything.
- A masked version (e.g. GB29 **** **** **** 1653) is stored separately so the frontend can display it without decrypting — keeping decryption calls to a minimum.

NEW TABLE	`MemberBankDetails`

– BankDetailId int PK IDENTITY
– MemberId int FK → Members.MemberId
– BankName nvarchar(100) — e.g. 'Barclays', 'ING', 'Revolut'
– AccountHolderName nvarchar(150) — full name on the account
– EncryptedIBAN nvarchar(500) — AES-256 CBC encrypted IBAN, stored as base64
– MaskedIBAN nvarchar(50) — e.g. 'GB29 **** **** **** 1653' — for display only, never used for transfer
– CreatedAt datetime2 default GETUTCDATE()
– UpdatedAt datetime2 NULL
– UNIQUE INDEX on MemberId — enforces one bank account per member. Use MERGE in sp_SaveBankDetails to update if exists.

NEW FILE	`Services/EncryptionService.cs`

– Purpose: provides AES-256 CBC encryption/decryption for IBAN values
– Encrypt(string plainText) → returns base64 string of the encrypted bytes. Uses Key + IV from config.
– Decrypt(string cipherText) → takes base64 ciphertext, returns plain IBAN string.

– MaskIBAN(string plain) → returns display-safe version, e.g. 'GB29 **** **** **** 1653'. Keep first 4 chars and last 4 chars visible.
– Key and IV are loaded from appsettings.Production.json under Encryption:Key and Encryption:IV
– Register in Program.cs: builder.Services.AddSingleton<EncryptionService>()
– FULL CODE is in FinShare_ForAbhilash_BankDetails.docx — copy it from there

CONFIG	**appsettings.Production.json (on server ONLY — never commit to Git)**

– Add under existing config: "Encryption": { "Key": "<32-byte base64>", "IV": "<16-byte base64>" }
– Generate Key: Convert.ToBase64String(RandomNumberGenerator.GetBytes(32))
– Generate IV: Convert.ToBase64String(RandomNumberGenerator.GetBytes(16))
– This file is already in .gitignore — safe to create on the server

NEW SP	**sp_SaveBankDetails**

– Params: @MemberId, @BankName, @AccountHolderName, @EncryptedIBAN, @MaskedIBAN
– Use MERGE: IF MemberId already has a row → UPDATE, ELSE → INSERT
– The C# controller calls EncryptionService.Encrypt(rawIBAN) and MaskIBAN(rawIBAN) BEFORE calling this SP

NEW SP	**sp_GetBankDetails**

– Params: @MemberId int
– Returns: BankName, AccountHolderName, MaskedIBAN — for the user's own profile page
– Note: return MaskedIBAN (not EncryptedIBAN) for display — the user does not need to see the full IBAN on their own profile

NEW SP	**sp_GetMemberBankDetailsForSettlement**

– Params: @SettlementId int, @RequestingUserId int
– Step 1: SELECT PayerMemberId, PayeeMemberId FROM Settlements WHERE SettlementId = @SettlementId
– Step 2: IF @RequestingUserId <> PayerMemberId → return empty result set immediately (403 enforced in controller)
– Step 3: IF @RequestingUserId = PayerMemberId → SELECT bank details of PayeeMemberId from MemberBankDetails
– Returns: BankName, AccountHolderName, EncryptedIBAN — controller will call EncryptionService.Decrypt() to show the full IBAN to the payer
– CRITICAL: This SP must NEVER return data if the caller is not the payer for THAT SPECIFIC settlement

NEW SP	**sp_DeleteBankDetails**

– Params: @MemberId int
– DELETE FROM MemberBankDetails WHERE MemberId = @MemberId
– Called when user removes their saved bank account

✏️ MODIFY	`Controllers/UserController.cs`

– GET /api/User/bank-details — calls sp_GetBankDetails. Returns BankName, AccountHolderName, MaskedIBAN. Requires [Authorize].

– PUT /api/User/bank-details — accepts SaveBankDetailsRequest { BankName, AccountHolderName, IBAN }. Validate IBAN format with regex. Call Encrypt(IBAN) and MaskIBAN(IBAN). Call sp_SaveBankDetails. Requires [Authorize].

– DELETE /api/User/bank-details — calls sp_DeleteBankDetails. Requires [Authorize].

✏️ MODIFY	`Controllers/SettlementController.cs`

– GET /api/Settlement/{id}/payee-bank-details — calls sp_GetMemberBankDetailsForSettlement with the settlement ID and the logged-in UserId.

– If SP returns empty → return 403 Forbidden ('You are not the payer for this settlement')

– If SP returns data → call EncryptionService.Decrypt(encryptedIBAN) to get the plain IBAN, return to the payer

– Requires [Authorize]

📌 **Note:** BankDetails.docx has the complete EncryptionService.cs code, all SQL with exact column names, and all controller methods. Read it before coding this phase.

PHASE 5 — Group Events Backend (Do Last)

Reference document: FinShare_ForAbhilash_EventBackend.docx

Why Group Events Need a Backend

Why this is needed:

• The frontend (Manus) has already built a group events feature. Users can create events like 'Trip to Amsterdam — 14 June' and see them under their group. However, right now events are saved to localStorage (the browser's built-in storage). This means events are DEVICE-SPECIFIC — if you create an event on your phone, nobody else in the group can see it. If you clear the browser, all events are gone.

• Moving events to the database fixes this completely. Once stored on the server, all group members see the same events on any device. This is the expected behaviour.

• The DB design includes IsDeleted for soft deletes — this means when an event is 'deleted' it is just hidden, not permanently removed. This is good practice for audit trails.

• Membership check in every SP is critical: a user who is NOT a member of the group must never be able to see or modify that group's events.

• Do this phase last because it requires both backend (this spec) AND frontend changes (Manus needs to switch from localStorage to API calls). Coordinate with Manus before starting.

NEW TABLE	`GroupEvents`

– EventId int PK IDENTITY
– GroupId int FK → Groups.GroupId
– CreatedByMemberId int FK → Members.MemberId
– Title nvarchar(200)
– Description nvarchar(500) NULL
– EventDate datetime2 — full date and time
– Location nvarchar(300) NULL
– CreatedAt datetime2 default GETUTCDATE()
– UpdatedAt datetime2 NULL
– IsDeleted bit default 0 — soft delete, do NOT hard delete events
– INDEX: IX_GroupEvents_GroupId on (GroupId) — fast lookup by group

NEW SP	`sp_GetGroupEvents`

– Params: @GroupId int, @RequestingUserId int
– Step 1: Verify @RequestingUserId is a member of @GroupId (JOIN GroupMembers). If not → return empty.
– Step 2: SELECT WHERE GroupId = @GroupId AND IsDeleted = 0 ORDER BY EventDate ASC

NEW SP	sp_CreateGroupEvent

– Params: @GroupId, @CreatedByMemberId, @Title, @Description, @EventDate, @Location
– Step 1: Verify @CreatedByMemberId is a group member. If not → raise error.
– Step 2: INSERT and return the new EventId

NEW SP	sp_UpdateGroupEvent

– Params: @EventId, @RequestingUserId, @Title, @Description, @EventDate, @Location
– Verify @RequestingUserId = CreatedByMemberId for this EventId (only creator can edit)
– UPDATE the event fields, SET UpdatedAt = GETUTCDATE()
– Return the updated row

NEW SP	sp_DeleteGroupEvent

– Params: @EventId int, @RequestingUserId int
– Verify @RequestingUserId = CreatedByMemberId
– Soft delete: UPDATE IsDeleted = 1, UpdatedAt = GETUTCDATE()
– Do NOT hard delete — keeps audit trail

NEW FILE	Controllers/GroupEventsController.cs

– GET /api/Group/{groupId}/events — calls sp_GetGroupEvents. Requires [Authorize].
– POST /api/Group/{groupId}/events — calls sp_CreateGroupEvent. Requires [Authorize].
– PUT /api/Group/{groupId}/events/{eventId} — calls sp_UpdateGroupEvent. Requires [Authorize].
– DELETE /api/Group/{groupId}/events/{eventId} — calls sp_DeleteGroupEvent. Requires [Authorize].
– Full code is in FinShare_ForAbhilash_EventBackend.docx

📌 **Note:** After this backend is ready, tell Manus so he can update the frontend to call these API endpoints instead of using localStorage. Until then, the frontend event feature will continue working on localStorage (single device only).

Summary — All Pending Tasks at a Glance

Ph.	Area	Priority	What to Build
2A	Refresh Tokens	⭐ **NEXT**	RefreshTokens table + 4 SPs + RefreshTokenService.cs + /refresh and /revoke endpoints
2B	WebAuthn/Biometric	**Low priority**	Fido2NetLib + WebAuthnCredentials table + 3 SPs + 4 endpoints
3A	Edit Profile	**After 2A**	3 SPs (GetProfile, UpdateProfile, UpdatePhoto) + UserController + SixLabors.ImageSharp NuGet
3B	Change Password	**After 2A**	sp_ChangePassword + PUT /api/User/change-password in UserController
3C	Transaction History	**After 2A**	2 SPs (UNION ALL history + summary) + TransactionController with 2 endpoints
3D	Notifications	**After 2A**	Notifications table + 5 SPs + NotificationController + trigger calls in 3 existing controllers
4	Bank Details (IBAN)	**After Ph. 3**	MemberBankDetails table + EncryptionService.cs + 4 SPs + 3 UserController endpoints + Settlement EP
5	Group Events	**Last**	GroupEvents table + 4 SPs + GroupEventsController (4 endpoints) — coordinate with Manus

Reference Documents

Each document has complete SQL, C# code, and step-by-step instructions for that phase:

Document Name	What It Contains
FinShare_ForAbhilash_SecurityPhase2.docx	Refresh Tokens (full code) + WebAuthn (full code)
FinShare_ForAbhilash_NewFeatures.docx	Edit Profile, Change Password, Transaction History, Notifications — full code

FinShare_ForAbhilash_BankDetails.docx	Bank Details, EncryptionService.cs, Settlement integration — full code
FinShare_ForAbhilash_EventBackend.docx	Group Events — GroupEvents table, all 4 SPs, GroupEventsController.cs
FinShare_ForAbhilash_TrueStatus.docx	Confirmed live ✅ vs still pending ❌ — accurate current status of all features
FinShare_ForAbhilash_JWTSecretStorage.docx	How to store JWT secret and Encryption key safely on the server

Recommended Order of Work

Phase 2A (Refresh Tokens) → Phase 3 (New Features: Profile, Password, History, Notifications) → Phase 4 (Bank Details) → Phase 5 (Group Events Backend) → Phase 2B (WebAuthn — optional)